



UNIVERSIDAD NACIONAL DE CÓRDOBA
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES
CÁTEDRA DE REDES DE COMPUTADORAS

TRABAJO PRÁCTICO N° 1

“Conceptos de capa física y capa de enlace de datos.”

Grupo: “Los simuLANdores”

Alumnos:

Barrio, Rafael

Chesta, Santiago

Garay, Alexis Tomás

Guzmán Gonzalez, Pedro

Vera, Agustín

Martin, Agostina Rocio

Zucchella Paz, Valentino

Profesor:

Ing. Santiago Martin Henn

Resolución de consignas:

Objetivos:

- Interiorizar didácticamente conceptos de las capas de Enlace de Datos (2), Red (3) y Transporte (4)

Requisitos:

- Acceso a computadora con Internet.
- Software Wireshark y Packet Tracer.
- Bibliografía de la materia.

Consignas:

1) Vamos a empezar observando cómo se organiza la información dentro de una red local:

a) ¿Qué función cumple la capa de enlace dentro del modelo OSI? ¿Qué tipo de comunicación resuelve?

- La función que cumple la capa de enlace dentro del modelo OSI (capa 2) es proporcionar una transferencia fiable de datos sobre el enlace físico.

Sus funciones pueden incluir

Entramado: organiza el flujo de bits en bloques discretos llamados tramas

Control de errores: Detecta y/o corrige errores de bits introducidos por el ruido (usa códigos como el **CRC**)

Control de flujo: Evita que un transmisor rápido sature el buffer de un receptor lento

Control de acceso al medio(MAC): Si el canal es compartido, coordina quién tiene derecho a transmitir en cada instante para evitar choques entre las partes

b) ¿Qué es una dirección MAC? ¿En qué se diferencia de una dirección IP?

- La dirección MAC es un identificador, que representa a cualquier interfaz de red (NIC o adaptador) conectada a un medio físico compartible (computadora, smartphone, impresora o cada una de las interfaces individuales de un router)

MAC vs IP

La dirección MAC es permanente, viene grabada de fábrica en el adaptador de red, nunca cambia sin importar a qué red física se conecte el dispositivo. Es el DNI de los dispositivos de red. Mientras que la dirección IP es jerárquica y dinámica (consta de una parte de red y una de host). Cambia necesariamente cuando el dispositivo se mueve de una subred a otra para poder encaminar los paquetes. Funciona como el código postal

La dirección MAC solo sirve para direccionar tramas dentro de la misma subred local.

La dirección IP sirve para encaminar paquetes a través de múltiples redes interconectadas globalmente.

c) ¿Qué es una trama Ethernet? Identificar sus principales campos y explicar brevemente para qué sirve cada uno.

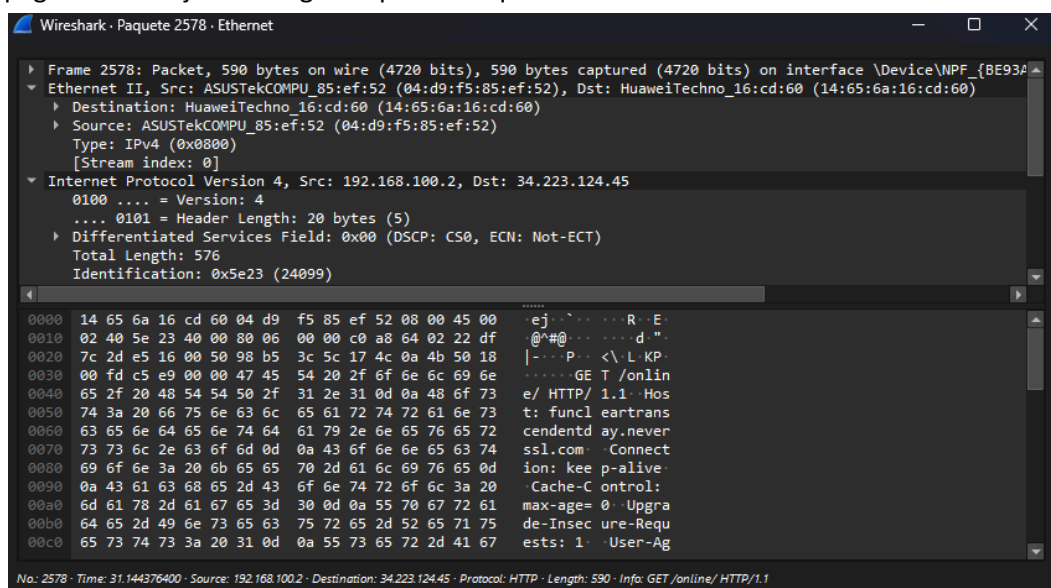
- Una trama Ethernet es la unidad de datos de la capa de enlace de datos (capa 2 del modelo OSI). Organiza la información en un formato estructurado, con cabecera, carga útil y cola para asegurar que los paquetes viajen sin errores entre equipos de una red local (LAN)
Su estructura estándar contiene los siguientes campos:

1. **Preámbulo(8 bytes):** Consiste en un patrón alternado de bits para sincronizar el reloj del receptor con el del emisor. El último byte termina en 10101011 para alertar del inicio de la trama
2. **Dirección MAC de destino (6 bytes):** Contiene la dirección física de la interfaz del nodo receptor local
3. **Dirección MAC de origen (6 bytes):** Contiene la dirección física de la interfaz del adaptador que transmite la trama
4. **Tipo/Longitud (2 bytes):** Especifica la longitud de los datos
5. **Datos (Payload / Datos - 46 a 1500 bytes):** Contiene el paquete encapsulado de la capa superior. Si el paquete mide menos de 46 bytes, se añade un campo de relleno (Padding) para alcanzar la longitud mínima de trama requerida para que el mecanismo de colisiones funcione correctamente
6. **Secuencia de comprobación de trama / CRC (4 bytes):** Código de redundancia cíclica calculado sobre la trama para permitir que el receptor detecte si hubo errores de bit durante la transmisión física y descartar la trama si detecta que fue corrupta.

d) ¿Qué información permite determinar qué protocolo de capa superior está transportando una trama Ethernet?

- La información la proporciona el campo **Tipo (EtherType)** de la trama Ethernet. Este campo de 2 bytes le indica a la tarjeta adaptadora en el destino a que protocolo específico de la capa superior debe entregar los datos encapsulados.
- **Códigos estandarizados comunes:**
 1. **0x0800** en hexadecimal indica que transporta un **datagrama IPv4**
 2. **0x0806** en hexadecimal indica que transporta un mensaje del **protocolo ARP**
 3. **0x86DD** indica que transporta un paquete **IPv6**.
 4. **0x8100** indica que se trata de una trama con etiqueta **VLAN 802.1Q**

2) Usando Wireshark, capturar tráfico generado por su propia computadora mientras acceden a una página web o ejecutan alguna aplicación que utilice la red.



a) Seleccionar una trama Ethernet e identificar las direcciones MAC de origen y destino. ¿A qué

¿Qué dispositivos creen que corresponden?

- **Direcciones MAC**

1. **Origen: ASUSTekCOMPU_85:ef:52** -> corresponde a la placa de red de la computadora
Destino: HuaweiTechno_16:cd:60 -> Dispositivo Huawei (router local)

b) Dentro de la misma trama, identificar el paquete IP. ¿Cuáles son las direcciones IP de origen y destino? (no importa si son versión 4 o versión 6)

- Dirección IP de origen: **192.168.100.2** (PC)
Dirección IP de destino: **34.223.124.45** (IP publica del servidor web remoto)

c) Comparar las direcciones MAC y las direcciones IP encontradas. ¿Representan lo mismo?

- La IP de destino es la del servidor real, en algún lugar de internet. Que sirve para encaminar el paquete extremo a extremo a través de múltiples routers por todo el internetwork global. Mientras que la dirección MAC es la del router local
- Esto confirma el concepto: la IP es de **extremo a extremo** (end-to-end, mismo origen y destino durante todo el viaje), mientras que la MAC es de **salto a salto** (hop-to-hop, cambia en cada router que atraviesa el paquete).

d) Observar el campo EtherType. ¿Qué protocolo está encapsulado dentro de la trama analizada?

- Observamos que el protocolo que está encapsulado es de **Type: IPv4 (0x0800)**, indica que dentro de la trama ethernet viaja un paquete IPv4

3) Vamos ahora a subir una capa y observar el transporte de información mediante TCP.

a) ¿Qué problema(s) resuelve TCP que no resuelve directamente Ethernet ni IP?

- TCP (Transmission Control Protocol) se ejecuta en los sistemas finales para resolver las siguientes deficiencias del canal físico y de red
 1. **Pérdida de datos (falta de fiabilidad):** Si un router intermedio descarta un paquete por congestión, IP no se entera ni lo recupera. TCP implementa el mecanismo PAR (Positive Acknowledgement with Retransmission): exige confirmaciones (ACKs) por cada octeto de datos recibido y utiliza temporizadores de retransmisión para reenviar automáticamente lo que se pierda en el camino
 2. **Desorden en la entrega:** Debido a que IP encamina cada paquete de manera independiente, estos pueden tomar rutas distintas y llegar desordenados. **TCP** asigna números de secuencia a cada byte transmitido, permitiendo al receptor reensamblar el flujo original en el orden correcto y descartar duplicados antes de entregarlo a la aplicación
 3. **Saturación del receptor(control de flujo):** IP y Ethernet envían datos sin importar si el receptor tiene espacio en memoria para procesarlos. **TCP** implementa un mecanismo de ventana deslizante (Sliding Window) mediante el cual el receptor le informa dinámicamente al emisor cuánto espacio libre le queda en su buffer (ventana de recepción), regulando la velocidad de transmisión para evitar desbordamientos

4. **Saturación de la red (control de congestión):** Si muchos hosts transmiten a la vez, los routers colapsan. TCP detecta pérdidas y demoras en el viaje extremo a extremo y ajusta dinámicamente su tasa de envío (ventana de congestión) para no agravar el estado de congestión del núcleo de la red

b) Investigar los campos más importantes de la metadata en un frame TCP. ¿Para qué sirve cada uno?

- Campos más importantes de la metadata de un frame TCP:
 - **Source Port / Destination Port:** Identifican a las aplicaciones
 - **Sequence Number (Número de Secuencia):** El identificador del byte que se está enviando. Sirve para rearmar los datos
 - **Acknowledgment Number (ACK):** Le dice al otro lado que ya recibió todo hasta el byte X, y que espera al siguiente
 - **Flags:** Son bits que indican el “estado” del paquete. Los más importantes son:
 - **SYN(Synchronize):** Avisa que quiere iniciar una conexión
 - **ACK(Acknowledgment):** Confirmación de recepción.
 - **FIN(Finish):** Confirmación de finalización de envío de datos.
 - **PSH(Push):** Enviar los datos ahora, suele estar acompañado de carga útil
 - **Window Size:** Es el control de flujo. Le dice al emisor que cantidad de bytes puede soportar

c) Explicar el Three y Four way handshake en TCP.

- **Three-way Handshake (Establecimiento de conexión):**
 - **Cliente(SYN):** El cliente manda un paquete con el flag SYN en 1 (Preguntando si está disponible para conectar)
 - **Servidor(SYN + ACK):** El servidor responde con ambos flags en 1. (el SYN Dice que si esta disponible para conectarse)(El ACK confirma la recepción del pedido de conexión)
 - **Cliente (ACK):** El cliente confirma. (Se confirma la conexión y pueden empezar a comunicarse)
- **Four-way Handshake (Terminando la conexión):**
 - **Cliente(FIN):** Le comunica al servidor que no hay nada más que enviar.
 - **Servidor(ACK):** Confirma que recibió el aviso del cliente.
 - **Servidor(FIN):** Le comunica al cliente que no tiene nada más para enviarle.
 - **Cliente(ACK):** Confirma la recepción del FIN.

El “Four-way Handshake” se llama así porque necesita 4 pasos, debido a que el servidor puede seguir enviando datos que le quedaron antes de finalizar la conexión. A diferencia del “Three-way Handshake” que solo necesita 3 pasos para conectarse de manera ordenada.

Iniciar una instancia de PacketSender (ésta será el servidor). Explorar la interfaz, discutir con los compañeros que

puede ser cada una de las cosas que estamos viendo.

En esta instancia, iniciar un servidor TCP:

¡Tomar nota del puerto!

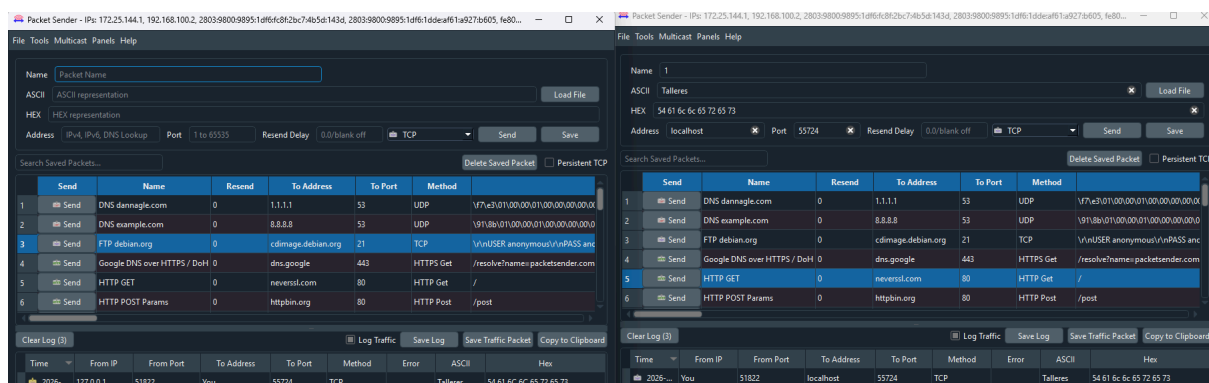
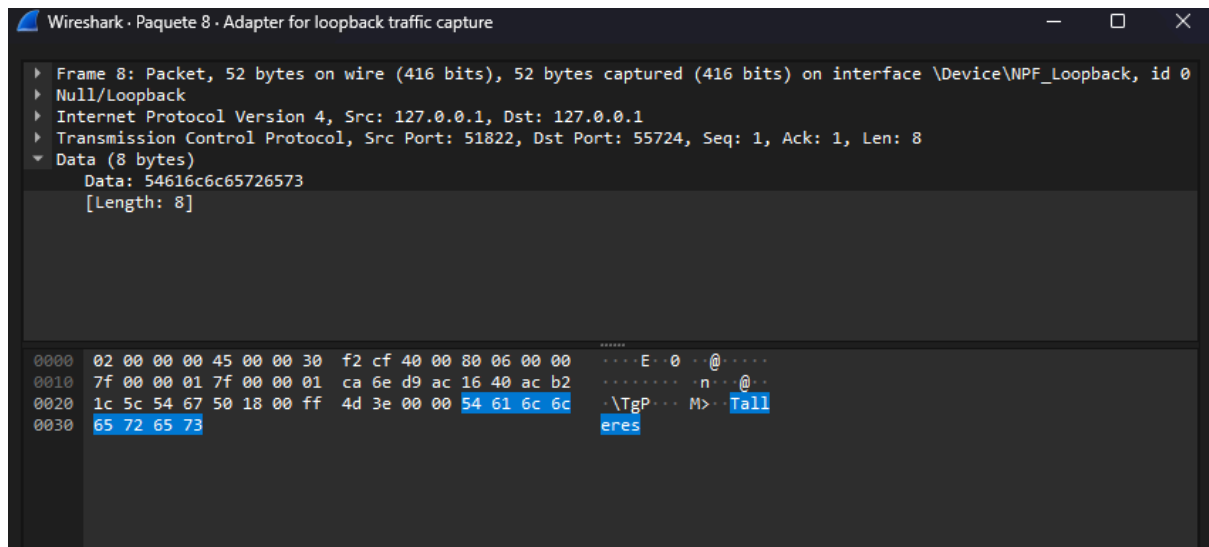
Iniciar otra instancia de PacketSender (cliente) sin cerrar la primera, configurar la dirección local (localhost,

127.0.0.1 o tu dirección IP si la conoces) y el puerto del servidor TCP que acabas de configurar:

Configurar wireshark para “sniffear” la conexión que vamos a hacer. Monitorear la dirección de loopback y, te

recomendamos, configurá algún filtro para tener la consola “limpia” hasta que iniciemos la conexión. Por

ejemplo, tcp.dstport == <puerto>.



d) Iniciar la conexión enviando un paquete, capturar el handshake y el paquete de datos. Analizar el paquete de datos, sus distintas partes y encontrar la carga útil del paquete usando WireShark.

- Enviamos el paquete, lo capturamos con wireshark. Analizamos e identificamos las partes y por último localizamos la carga útil (payload) - El mensaje que enviamos fue “Talleres”

e) Finalizar la conexión y capturar el Four-way handshake.

10	2.446341000	127.0.0.1	127.0.0.1	TCP	44	55724 → 51822	[FIN, ACK]	Seq=1 Ack=9 Win=65280 Len=0
11	2.446363800	127.0.0.1	127.0.0.1	TCP	44	51822 → 55724	[ACK]	Seq=9 Ack=2 Win=65280 Len=0
12	2.446393300	127.0.0.1	127.0.0.1	TCP	44	51822 → 55724	[FIN, ACK]	Seq=9 Ack=2 Win=65280 Len=0
13	2.446426900	127.0.0.1	127.0.0.1	TCP	44	55724 → 51822	[ACK]	Seq=2 Ack=10 Win=65280 Len=0

[FIN, ACK] ← el cliente dice "ya no tengo más para enviar"

[ACK] ← el servidor confirma que recibió el FIN

[FIN, ACK] ← el servidor también dice "yo tampoco tengo más para enviar"

[ACK] ← el cliente confirma el FIN del servidor

4 paquetes porque, como TCP es full-duplex, cada extremo tiene que cerrar su propio canal de subida de datos por separado (primero cierra uno, se confirma; después cierra el otro, se confirma).

f) ¿Qué conclusión podemos sacar de que sea tan fácil ver un paquete que viaja a través de la red?

- Se comprueba que TCP (y las capas inferiores, IP y Ethernet) **no ofrecen ningún tipo de cifrado ni protección de la información por diseño**. En nuestra captura pudimos ver literalmente el texto "Talleres" viajando en claro dentro del payload del paquete, visible con solo abrir Wireshark, una herramienta gratuita y accesible para cualquiera. Esto significa que **cualquier persona con acceso al medio por donde circula el tráfico** (una red Wi-Fi compartida, un switch intermedio, un proveedor de internet, etc.) podría capturar y leer contraseñas, mensajes o datos sensibles enviados sin cifrado. Es precisamente por esto que existen protocolos de la capa de aplicación/transporte como **TLS/SSL** (el que usa HTTPS), que cifra la carga útil para que, aunque alguien capture el paquete, no pueda leer su contenido sin la clave de descifrado correspondiente.

4) Vamos a iniciar un cliente en packet sender y comunicarnos con el servidor que nos indique el profe (los datos van a estar en el drive compartido). Este será una dirección IP y un puerto para un servicio montado en la nube.

El servidor responderá a comandos que finalicen con la secuencia \r (este es un caracter especial que pueden ingresar en el campo de carga útil ASCII así tal cual, barra invertida erre).

Iniciar una conexión con el servidor y obtener respuestas para comandos admitidos: hola, ping, tic, status.

Capturar toda la sesión de conexión con WireShark. Podés seleccionar la opción "Persistent TCP":

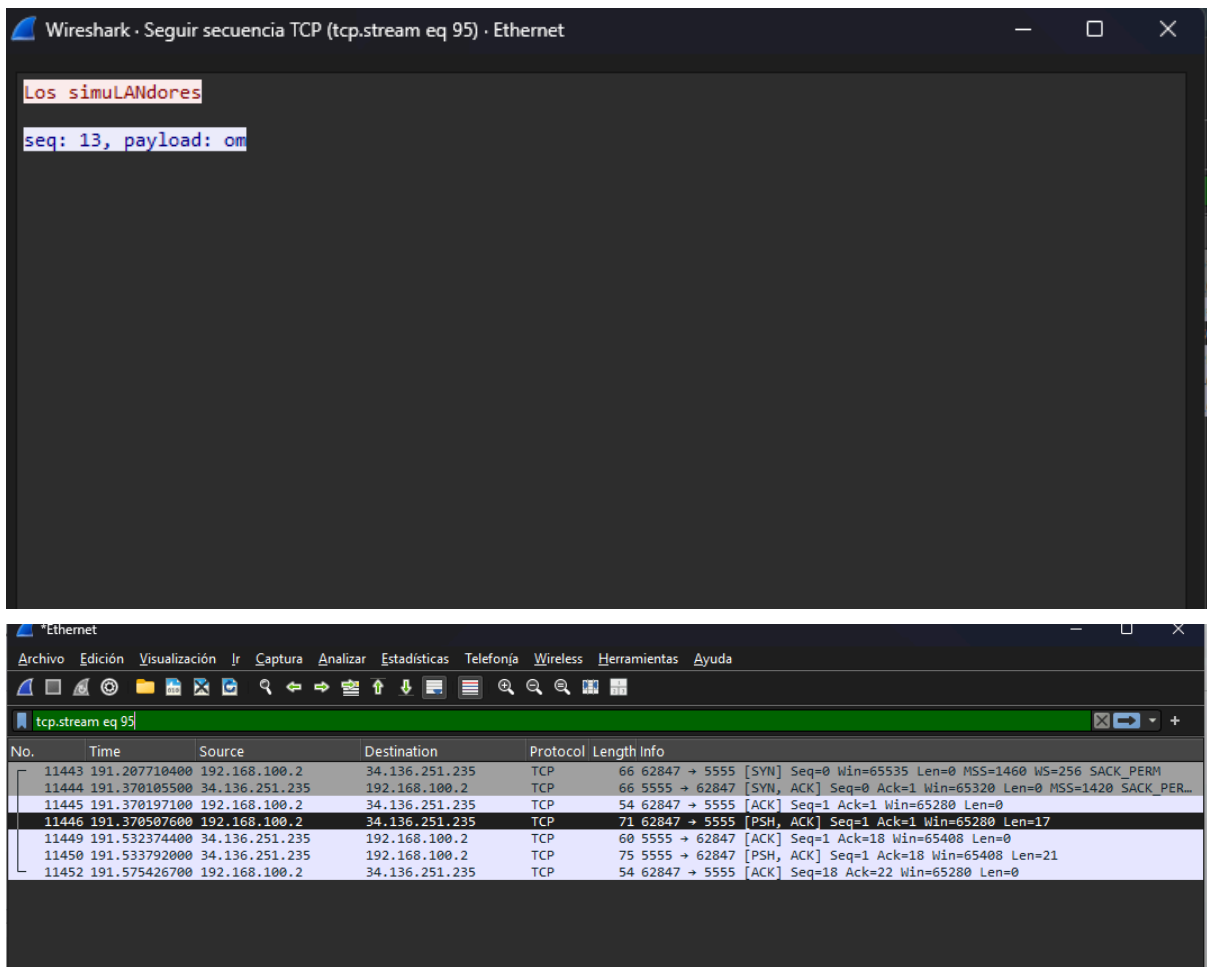
Para no finalizar la conexión, se abrirá una ventana donde podrás interactuar (cual chat) con el server:

Acá tenés la opción "Append \r" así no tenes que estar poniendo manualmente el caracter cada que envías mensajes.

Para finalizar el TP:

El servidor está configurado para que los nombres de los grupos sean comandos válidos. Iniciar la conexión, enviar el comando correspondiente a su grupo y documentar la respuesta del servidor, tanto en el informe como en la pestaña correspondiente en la planilla compartida de Drive.

Comportamiento esperado al interactuar con el server:



Al establecer una conexión TCP con el servidor remoto (**34.136.251.235:5555**) pudimos comprobar en la práctica cómo funciona un protocolo simple a nivel de aplicación construido sobre TCP: el cliente envía comandos de texto plano finalizados en `\r`, y el servidor responde de forma sincrónica sobre la misma conexión. Gracias a Wireshark, y en particular a la función Follow TCP Stream, pudimos reconstruir toda la conversación completa entre cliente y servidor de forma legible, sin ningún esfuerzo adicional, ya que —al igual que en el punto 3— el tráfico viaja sin cifrar. Al enviar el comando correspondiente a nuestro grupo (**Los simuLANDores**), el servidor respondió con **seq: 13, payload: om**, confirmando la recepción del comando y devolviendo un dato de control asociado a esa sesión.

Esto refuerza la conclusión general del TP: TCP garantiza que los datos lleguen completos, en orden y sin errores (a diferencia de IP o Ethernet, que no aseguran nada de eso), pero no aporta ningún tipo de seguridad ni confidencialidad. Cualquier intermediario en la red (o incluso nosotros mismos, sniffando nuestro propio tráfico) puede ver exactamente qué se está enviando y recibiendo. En un entorno real, este mismo intercambio debería viajar cifrado para evitar que la información (en este caso el comando del grupo, en otros casos podrían ser credenciales o datos sensibles) quede expuesta a cualquiera que esté capturando el tráfico de la red.